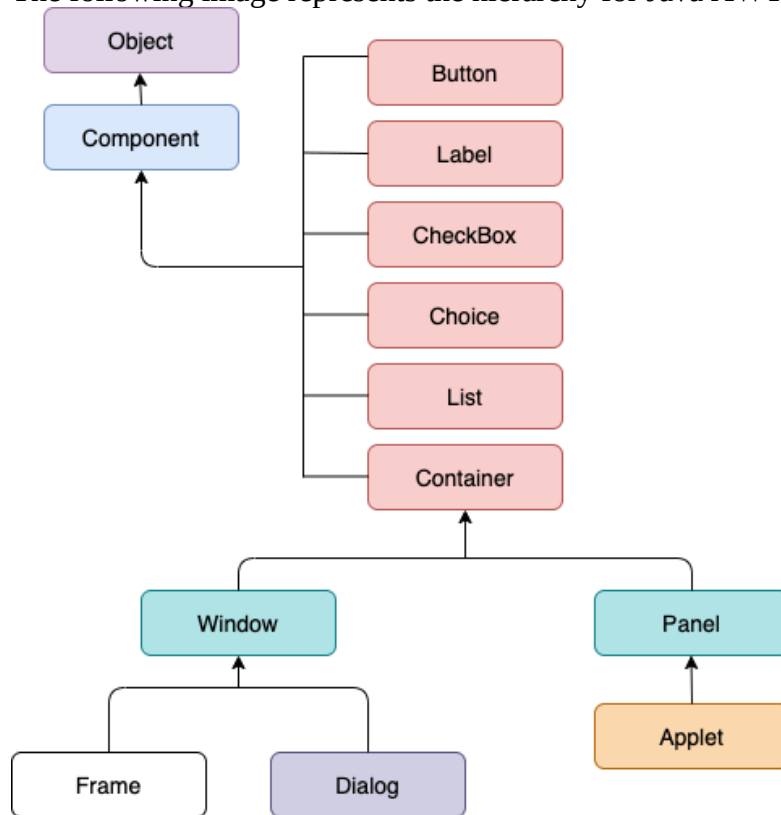


AWT - Working with windows using AWT Classes:

AWT stands for Abstract window toolkit is an Application programming interface (API) for creating Graphical User Interface (GUI) in Java. It allows Java programmers to develop window-based applications.

AWT provides various components like button, label, checkbox, etc. used as objects inside a Java Program. AWT components use the resources of the operating system, i.e., they are platform-dependent, which means, component's view can be changed according to the view of the operating system. The classes for AWT are provided by the Java.awt package for various AWT components. The following image represents the hierarchy for Java AWT.

**Component Class**

The component class stands at the top of the AWT hierarchy, is an abstract class that contains all the properties of the component visible on the screen. The Component object contains information about the currently selected foreground and background color. It also has information about the currently selected text color.

Container

The container is a component that contains other components like button, textfield, label, etc. However, it is a subclass of the Component class.

Panel

The panel can be defined as a container that can be used to hold other components. However, it doesn't contain the title bar, menu bar, or border.

Window

A window can be defined as a container that doesn't contain any border or menu bar. It creates a top-level view. However, we must have a frame, dialog, or another window for creating a window.

Frame

The frame is a subclass of Window. It can be defined as a container with components like button, textfield, label, etc. In other words, AWT applications are mostly created using frame container.

Java AWT Example

Consider the following simple example of Java AWT in which we have shown an awt component button by setting its placement and window frame size.

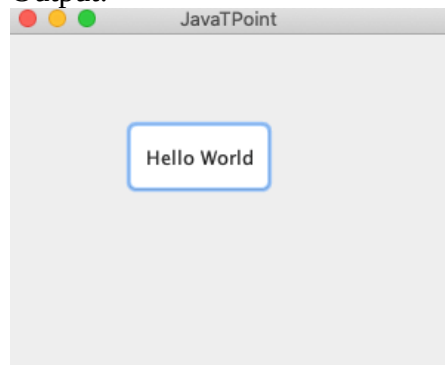
```
import java.awt.*;

public class AwtProgram1 {
    public AwtProgram1()
    {
        Frame f = new Frame();
        Button btn=new Button("Hello World");
        btn.setBounds(80, 80, 100, 50);
        f.add(btn);    //adding a new Button.
        f.setSize(300, 250);    //setting size.
        f.setTitle("JavaTPoint"); //setting title.
        f.setLayout(null); //set default layout for frame.
        f.setVisible(true);    //set frame visibility true.
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        AwtProgram1 awt = new AwtProgram1(); //creating a frame.
    }
}
```

Output:



Java awt Example (extending Frame Class)

Consider the following program in which we have created a user's form GUI, which has three fields, i.e., first name, last name, and date of birth.

```
import java.awt.*;
public class AwtApp extends Frame {

    AwtApp(){
        Label firstName = new Label("First Name");
        firstName.setBounds(20, 50, 80, 20);
```

```
Label lastName = new Label("Last Name");
lastName.setBounds(20, 80, 80, 20);
```

```
Label dob = new Label("Date of Birth");
dob.setBounds(20, 110, 80, 20);
```

```
TextField firstNameTF = new TextField();
firstNameTF.setBounds(120, 50, 100, 20);
```

```
TextField lastNameTF = new TextField();
lastNameTF.setBounds(120, 80, 100, 20);
```

```
TextField dobTF = new TextField();
dobTF.setBounds(120, 110, 100, 20);
```

```
Button sbmt = new Button("Submit");
sbmt.setBounds(20, 160, 100, 30);
```

```
Button reset = new Button("Reset");
reset.setBounds(120,160,100,30);
```

```
add(firstName);
add(lastName);
add(dob);
add(firstNameTF);
add(lastNameTF);
add(dobTF);
add(sbmt);
add(reset);
```

```
setSize(300,300);
setLayout(null);
setVisible(true);
}
```

```
public static void main(String[] args) {
// TODO Auto-generated method stub
AwtApp awt = new AwtApp();
}
}
```

Output:

Java AWT Checkbox

The Checkbox class is used to create a checkbox. It is used to turn an option on (true) or off (false). Clicking on a Checkbox changes its state from "on" to "off" or from "off" to "on". AWT Checkbox Class Declaration

public class Checkbox **extends** Component **implements** ItemSelectable, Accessible

Checkbox Class Constructors

Sr. no.	Constructor	Description
1.	Checkbox()	It constructs a checkbox with no string as the label.
2.	Checkbox(String label)	It constructs a checkbox with the given label.
3.	Checkbox(String label, boolean state)	It constructs a checkbox with the given label and sets the given state.
4.	Checkbox(String label, boolean state, CheckboxGroup group)	It constructs a checkbox with the given label, set the given state in the specified checkbox group.
5.	Checkbox(String label, CheckboxGroup group, boolean state)	It constructs a checkbox with the given label, in the given checkbox group and set to the specified state.

Java AWT Checkbox Example

In the following example we are creating two checkboxes using the Checkbox(String label) constructo and adding them into the Frame using add() method.

CheckboxExample1.java

```
// importing AWT class
import java.awt.*;
public class CheckboxExample1
{
// constructor to initialize
    CheckboxExample1() {
// creating the frame with the title
```

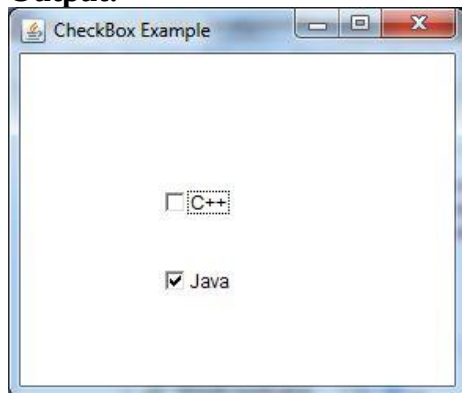
```

    Frame f = new Frame("Checkbox Example");
// creating the checkboxes
    Checkbox checkbox1 = new Checkbox("C++");
    checkbox1.setBounds(100, 100, 50, 50);
    Checkbox checkbox2 = new Checkbox("Java", true);
// setting location of checkbox in frame
    checkbox2.setBounds(100, 150, 50, 50);
// adding checkboxes to frame
    f.add(checkbox1);
    f.add(checkbox2);

// setting size, layout and visibility of frame
    f.setSize(400,400);
    f.setLayout(null);
    f.setVisible(true);
}
// main method
public static void main (String args[])
{
    new CheckboxExample1();
}
}

```

Output:



Java AWT Choice

The object of Choice class is used to show popup menu of choices. Choice selected by user is shown on the top of a menu. It inherits Component class.

AWT Choice Class Declaration

public class Choice **extends** Component **implements** ItemSelectable, Accessible

Choice Class constructor

Sr. no.	Constructor	Description
1.	Choice()	It constructs a new choice menu.

Methods inherited by class

The methods of Choice class are inherited by following classes:

java.awt.Component

java.lang.Object

Java AWT Choice Example

In the following example, we are creating a choice menu using Choice() constructor. Then we add 5 items to the menu using add() method and Then add the choice menu into the Frame.

ChoiceExample1.java

// importing awt class

import java.awt.*;

public class ChoiceExample1 {

 // class constructor

 ChoiceExample1() {

 // creating a frame

 Frame f = **new** Frame();

 // creating a choice component

 Choice c = **new** Choice();

 // setting the bounds of choice menu

 c.setBounds(100, 100, 75, 75);

 // adding items to the choice menu

 c.add("Item 1");

 c.add("Item 2");

 c.add("Item 3");

 c.add("Item 4");

 c.add("Item 5");

 // adding choice menu to frame

 f.add(c);

 // setting size, layout and visibility of frame

 f.setSize(400, 400);

 f.setLayout(**null**);

 f.setVisible(**true**);

 }

// main method

public static void main(String args[])

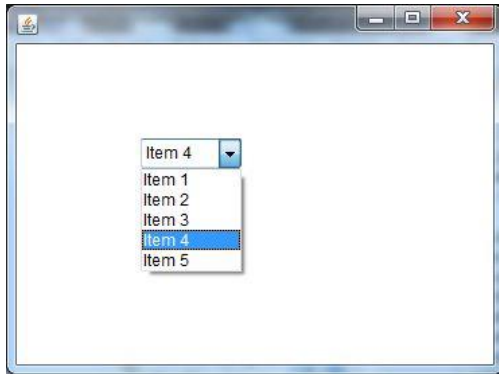
{

new ChoiceExample1();

}

}

Output:



Java AWT List Example

In the following example, we are creating a List component with 5 rows and adding it into the Frame.

ListExample1.java

// importing awt class

import java.awt.*;

public class ListExample1

{

 // class constructor

 ListExample1() {

 // creating the frame

 Frame f = **new** Frame();

 // creating the list of 5 rows

 List l1 = **new** List(5);

 // setting the position of list component

 l1.setBounds(100, 100, 75, 75);

 // adding list items into the list

 l1.add("Item 1");

 l1.add("Item 2");

 l1.add("Item 3");

 l1.add("Item 4");

 l1.add("Item 5");

 // adding the list to frame

 f.add(l1);

 // setting size, layout and visibility of frame

 f.setSize(400, 400);

 f.setLayout(**null**);

 f.setVisible(**true**);

 }

 // main method

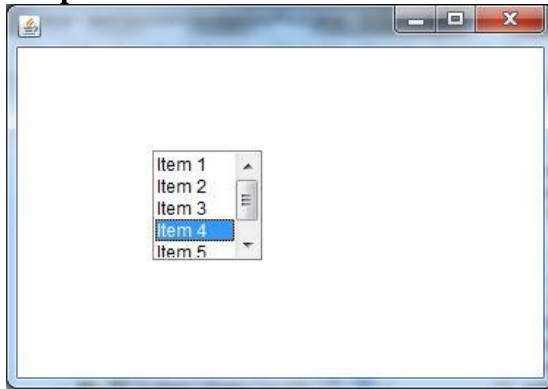
public static void main(String args[])

{

new ListExample1();

} }

Output:



Some more Components of AWT

1. **Layout Managers:** AWT includes layout managers like FlowLayout, BorderLayout, and GridLayout that help in arranging components within containers.
2. **Event Handling:** AWT supports event handling for user interactions. You can use event listeners and adapters to handle events like button clicks, mouse movements, and keypresses.
3. **Containers:** AWT provides container classes like Frame, Panel, and Window for organizing and grouping components.
4. **Graphics:** You can draw shapes, text, and images using the Graphics and Graphics2D classes.
5. **Fonts and Colors:** AWT allows you to work with fonts and colors for customizing the appearance of your components.

Some Advance topics of java AWT

1. Event Handling Mechanisms in Java AWT

Event handling in Java AWT is a crucial aspect of building interactive graphical user interfaces. It allows your program to respond to user actions, such as mouse clicks, key presses, window events, and more. Java AWT provides various event listener interfaces and adapters to handle events effectively. Here, we'll discuss event handling mechanisms and provide an example to illustrate how to use them.

ActionListener: Handles action events, like button clicks.

MouseListener: Manages mouse events, such as clicks and movements.

KeyListener: Deals with keyboard events, like key presses and releases.

WindowListener: Listens to window-related events like opening, closing, or resizing.

ComponentListener: Handles component-specific events, e.g., resizing components.

Example of Event Handling with ActionListener

Here we'll focus on event handling code using the ActionListener interface, which is commonly used for button clicks. We'll create a simple AWT application with a button, and when the button is clicked, a message will be displayed.

Java

```
import java.awt.*;
import java.awt.event.*;

public class GFG {
    public static void main(String[] args) {
        Frame frame = new Frame("Event Handling Example");
```

```

Button button = new Button("Click Me");

// Add the button to the frame
frame.add(button);

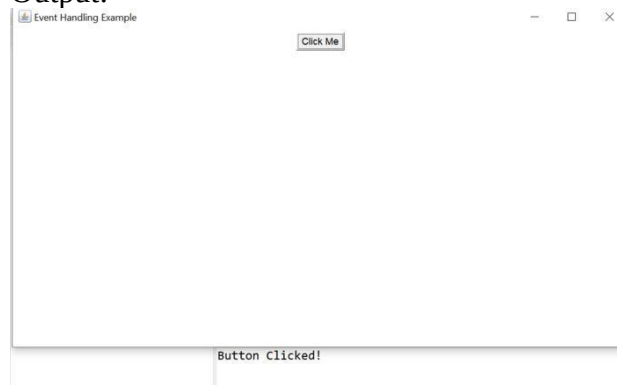
// Register an ActionListener for the button
button.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        // Code to execute when the button is clicked
        System.out.println("Button Clicked!");
    }
});

// Set layout manager and window properties
frame.setLayout(new FlowLayout());
frame.setSize(300, 100);
frame.setVisible(true);

// Handle window close event
frame.addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        System.exit(0);
    }
});
}
}

```

Output:



2. Double Buffering

Double buffering is a technique used in Java AWT and other graphical environments to improve the smoothness and flicker-free rendering of images, animations, and other graphical content. It involves using two buffers, one for drawing and one for displaying. This technique is particularly useful in situations where flickering and visual artifacts need to be minimized.

How Double Buffering Works?

Front Buffer : This is the buffer that is currently displayed on the screen. It's what the user sees. Any drawing directly onto the front buffer is immediately visible on the screen.

Back Buffer : This is an off-screen buffer that is not visible to the user. It's used for rendering and preparing the content to be displayed. Drawing operations are performed on the back buffer, and

when the content is ready, it's swapped with the front buffer to make it visible.

Advantages of Double Buffering

Smooth Animations: For animations, double buffering can provide smoother transitions between frames, enhancing the visual quality.

Avoiding Visual Artifacts: In situations where multiple elements are updated simultaneously (e.g., in games or real-time simulations), double buffering can prevent visual artifacts like "tearing."

Following is the list of commonly used controls while designed GUI using AWT.

Sr. No.	Control & Description
1	Label A Label object is a component for placing text in a container.
2	Button This class creates a labeled button.
3	Check Box A check box is a graphical component that can be in either an on (true) or off (false) state.
4	Check Box Group The CheckboxGroup class is used to group the set of checkbox.
5	List The List component presents the user with a scrolling list of text items.
6	Text Field A TextField object is a text component that allows for the editing of a single line of text.
7	Text Area A TextArea object is a text component that allows for the editing of a multiple lines of text.
8	Choice A Choice control is used to show pop up menu of choices. Selected choice is shown on the top of the menu.
9	Canvas A Canvas control represents a rectangular area where application can draw something or can receive inputs created by user.
10	Image An Image control is superclass for all image classes representing graphical images.
11	Scroll Bar A Scrollbar control represents a scroll bar component in order to enable user to select from range of values.

12	Dialog A Dialog control represents a top-level window with a title and a border used to take some form of input from the user.
13	File Dialog A FileDialog control represents a dialog window from which the user can select a file.

Java LayoutManagers

The LayoutManagers are used to arrange components in a particular manner. The Java LayoutManagers facilitates us to control the positioning and size of the components in GUI forms. LayoutManager is an interface that is implemented by all the classes of layout managers. There are the following classes that represent the layout managers:

```
java.awt.BorderLayout
java.awt.FlowLayout
java.awt.GridLayout
java.awt.CardLayout
java.awt.GridBagLayout
javax.swing.BoxLayout
javax.swing.GroupLayout
javax.swing.ScrollPaneLayout
javax.swing.SpringLayout etc.
```

Java BorderLayout:

The BorderLayout is used to arrange the components in five regions: north, south, east, west, and center. Each region (area) may contain one component only. It is the default layout of a frame or window. The BorderLayout provides five constants for each region:

```
public static final int NORTH
public static final int SOUTH
public static final int EAST
public static final int WEST
public static final int CENTER
```

Constructors of BorderLayout class:

BorderLayout(): creates a border layout but with no gaps between the components.

BorderLayout(int hgap, int vgap): creates a border layout with the given horizontal and vertical gaps between the components.

Example of BorderLayout class: Using BorderLayout() constructor

FileName: Border.java

```
import java.awt.*;
import javax.swing.*;
```

```
public class Border
{
    JFrame f;
    BorderLayout()
    {
        f = new JFrame();
```

```

// creating buttons
JButton b1 = new JButton("NORTH"); // the button will be labeled as NORTH
JButton b2 = new JButton("SOUTH"); // the button will be labeled as SOUTH
JButton b3 = new JButton("EAST"); // the button will be labeled as EAST
JButton b4 = new JButton("WEST"); // the button will be labeled as WEST
JButton b5 = new JButton("CENTER"); // the button will be labeled as CENTER

f.add(b1, BorderLayout.NORTH); // b1 will be placed in the North Direction
f.add(b2, BorderLayout.SOUTH); // b2 will be placed in the South Direction
f.add(b3, BorderLayout.EAST); // b2 will be placed in the East Direction
f.add(b4, BorderLayout.WEST); // b2 will be placed in the West Direction
f.add(b5, BorderLayout.CENTER); // b2 will be placed in the Center

f.setSize(300, 300);
f.setVisible(true);
}
public static void main(String[] args) {
    new Border();
} }
Output:

```



Example of BorderLayout class: Using BorderLayout(int hgap, int vgap) constructor

The following example inserts horizontal and vertical gaps between buttons using the parameterized constructor BorderLayout(int hgap, int vgap)

FileName: BorderLayoutExample.java

```

// import statement
import java.awt.*;
import javax.swing.*;
public class BorderLayoutExample
{
    JFrame jframe;
    // constructor
    BorderLayoutExample()
    {
        // creating a Frame
        jframe = new JFrame();
    }
}

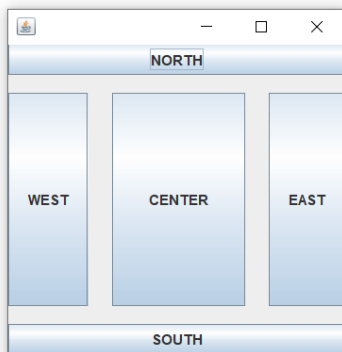
```

```

// create buttons
JButton btn1 = new JButton("NORTH");
JButton btn2 = new JButton("SOUTH");
JButton btn3 = new JButton("EAST");
JButton btn4 = new JButton("WEST");
JButton btn5 = new JButton("CENTER");
// creating an object of the BorderLayout class using
// the parameterized constructor where the horizontal gap is 20
// and vertical gap is 15. The gap will be evident when buttons are placed
// in the frame
jframe.setLayout(new BorderLayout(20, 15));
jframe.add(btn1, BorderLayout.NORTH);
jframe.add(btn2, BorderLayout.SOUTH);
jframe.add(btn3, BorderLayout.EAST);
jframe.add(btn4, BorderLayout.WEST);
jframe.add(btn5, BorderLayout.CENTER);
jframe.setSize(300,300);
jframe.setVisible(true);
}
// main method
public static void main(String args[])
{
    new BorderLayoutExample();
}
}

```

Output:



Java BorderLayout: Without Specifying Region

The add() method of the JFrame class can work even when we do not specify the region. In such a case, only the latest component added is shown in the frame, and all the components added previously get discarded. The latest component covers the whole area. The following example shows the same.

FileName: BorderLayoutWithoutRegionExample.java

```

// import statements
import java.awt.*;
import javax.swing.*;

public class BorderLayoutWithoutRegionExample
{

```

```

JFrame jframe;

// constructor
BorderLayoutWithoutRegionExample()
{
    jframe = new JFrame();

    JButton btn1 = new JButton("NORTH");
    JButton btn2 = new JButton("SOUTH");
    JButton btn3 = new JButton("EAST");
    JButton btn4 = new JButton("WEST");
    JButton btn5 = new JButton("CENTER");

    // horizontal gap is 7, and the vertical gap is 7
    // Since region is not specified, the gaps are of no use
    jframe.setLayout(new BorderLayout(7, 7));

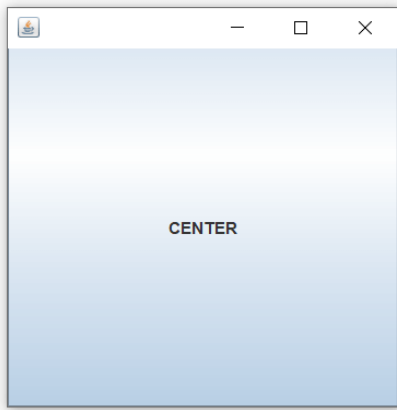
    // each button covers the whole area
    // however, the btn5 is the latest button
    // that is added to the frame; therefore, btn5
    // is shown
    jframe.add(btn1);
    jframe.add(btn2);
    jframe.add(btn3);
    jframe.add(btn4);
    jframe.add(btn5);

    jframe.setSize(300,300);
    jframe.setVisible(true);
}

// main method
public static void main(String args[])
{
    new BorderLayoutWithoutRegionExample();
}
}

```

Output:



Java GridLayout

The Java GridLayout class is used to arrange the components in a rectangular grid. One component is displayed in each rectangle.

Constructors of GridLayout class

GridLayout(): creates a grid layout with one column per component in a row.

GridLayout(int rows, int columns): creates a grid layout with the given rows and columns but no gaps between the components.

GridLayout(int rows, int columns, int hgap, int vgap): creates a grid layout with the given rows and columns along with given horizontal and vertical gaps.

Example of GridLayout class: Using GridLayout() Constructor

The GridLayout() constructor creates only one row. The following example shows the usage of the parameterless constructor.

FileName: GridLayoutExample.java

```
// import statements
```

```
import java.awt.*;
```

```
import javax.swing.*;
```

```
public class GridLayoutExample
```

```
{
```

```
    JFrame frameObj;
```

```
// constructor
```

```
    GridLayoutExample()
```

```
{
```

```
        frameObj = new JFrame();
```

```
// creating 9 buttons
```

```
        JButton btn1 = new JButton("1");
```

```
        JButton btn2 = new JButton("2");
```

```
        JButton btn3 = new JButton("3");
```

```
        JButton btn4 = new JButton("4");
```

```
        JButton btn5 = new JButton("5");
```

```
        JButton btn6 = new JButton("6");
```

```
        JButton btn7 = new JButton("7");
```

```
        JButton btn8 = new JButton("8");
```

```
        JButton btn9 = new JButton("9");
```

```

// adding buttons to the frame
// since, we are using the parameterless constructor, therefore;
// the number of columns is equal to the number of buttons we
// are adding to the frame. The row count remains one.
frameObj.add(btn1); frameObj.add(btn2); frameObj.add(btn3);
frameObj.add(btn4); frameObj.add(btn5); frameObj.add(btn6);
frameObj.add(btn7); frameObj.add(btn8); frameObj.add(btn9);

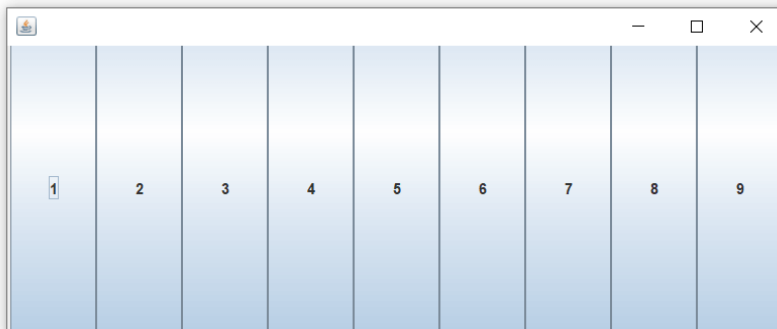
// setting the grid layout using the parameterless constructor
frameObj.setLayout(new GridLayout());

frameObj.setSize(300, 300);
frameObj.setVisible(true);
}

// main method
public static void main(String argsv[])
{
new GridLayoutExample();
}
}

```

Output:



Example of GridLayout class: Using GridLayout(int rows, int columns) Constructor

FileName: MyGridLayout.java

```

import java.awt.*;
import javax.swing.*;
public class MyGridLayout{
JFrame f;
MyGridLayout(){
    f=new JFrame();
    JButton b1=new JButton("1");
    JButton b2=new JButton("2");
    JButton b3=new JButton("3");
    JButton b4=new JButton("4");
    JButton b5=new JButton("5");
    JButton b6=new JButton("6");

```

```

JButton b7=new JButton("7");
JButton b8=new JButton("8");
JButton b9=new JButton("9");
// adding buttons to the frame
f.add(b1); f.add(b2); f.add(b3);
f.add(b4); f.add(b5); f.add(b6);
f.add(b7); f.add(b8); f.add(b9);

// setting grid layout of 3 rows and 3 columns
f.setLayout(new GridLayout(3,3));
f.setSize(300,300);
f.setVisible(true);
}
public static void main(String[] args) {
    new MyGridLayout();
}
}

```

Output:



Example of GridLayout class: Using GridLayout(int rows, int columns, int hgap, int vgap)

Constructor

The following example inserts horizontal and vertical gaps between buttons using the parameterized constructor GridLayout(int rows, int columns, int hgap, int vgap).

FileName: GridLayoutExample1.java

// import statements

import java.awt.*;

import javax.swing.*;

```

public class GridLayoutExample1
{

```

JFrame frameObj;

// constructor

GridLayoutExample1()

```

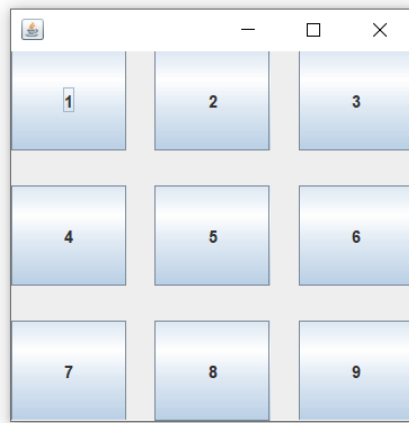
{
frameObj = new JFrame();

// creating 9 buttons
JButton btn1 = new JButton("1");
JButton btn2 = new JButton("2");
JButton btn3 = new JButton("3");
JButton btn4 = new JButton("4");
JButton btn5 = new JButton("5");
JButton btn6 = new JButton("6");
JButton btn7 = new JButton("7");
JButton btn8 = new JButton("8");
JButton btn9 = new JButton("9");

// adding buttons to the frame
// since, we are using the parameterless constructor, therefore;
// the number of columns is equal to the number of buttons we
// are adding to the frame. The row count remains one.
frameObj.add(btn1); frameObj.add(btn2); frameObj.add(btn3);
frameObj.add(btn4); frameObj.add(btn5); frameObj.add(btn6);
frameObj.add(btn7); frameObj.add(btn8); frameObj.add(btn9);
// setting the grid layout
// a 3 * 3 grid is created with the horizontal gap 20
// and vertical gap 25
frameObj.setLayout(new GridLayout(3, 3, 20, 25));
frameObj.setSize(300, 300);
frameObj.setVisible(true);
}
// main method
public static void main(String argsv[])
{
new GridLayoutExample();
}
}

```

Output:



Java FlowLayout

The Java FlowLayout class is used to arrange the components in a line, one after another (in a flow). It is the default layout of the applet or panel.

Fields of FlowLayout class

public static final int LEFT

public static final int RIGHT

public static final int CENTER

public static final int LEADING

public static final int TRAILING

Constructors of FlowLayout class

FlowLayout(): creates a flow layout with centered alignment and a default 5 unit horizontal and vertical gap.

FlowLayout(int align): creates a flow layout with the given alignment and a default 5 unit horizontal and vertical gap.

FlowLayout(int align, int hgap, int vgap): creates a flow layout with the given alignment and the given horizontal and vertical gap.

Example of FlowLayout class: Using FlowLayout() constructor

FileName: FlowLayoutExample.java

```
// import statements
```

```
import java.awt.*;
```

```
import javax.swing.*;
```

```
public class FlowLayoutExample  
{
```

```
    JFrame frameObj;
```

```
    // constructor
```

```
    FlowLayoutExample()  
{
```

```
        // creating a frame object
```

```
        frameObj = new JFrame();
```

```
        // creating the buttons
```

```
        JButton b1 = new JButton("1");
```

```
        JButton b2 = new JButton("2");
```

```
        JButton b3 = new JButton("3");
```

```
        JButton b4 = new JButton("4");
```

```
        JButton b5 = new JButton("5");
```

```
        JButton b6 = new JButton("6");
```

```
        JButton b7 = new JButton("7");
```

```
        JButton b8 = new JButton("8");
```

```
        JButton b9 = new JButton("9");
```

```
        JButton b10 = new JButton("10");
```

```
        // adding the buttons to frame
```

```
        frameObj.add(b1); frameObj.add(b2); frameObj.add(b3); frameObj.add(b4);
```

```
        frameObj.add(b5); frameObj.add(b6); frameObj.add(b7); frameObj.add(b8);
```

```
        frameObj.add(b9); frameObj.add(b10);
```

```

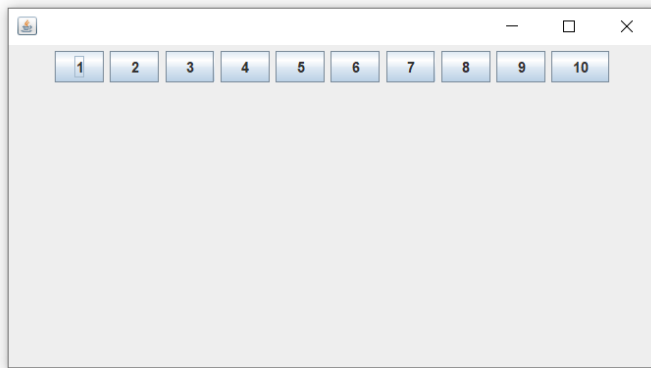
// parameter less constructor is used
// therefore, alignment is center
// horizontal as well as the vertical gap is 5 units.
frameObj.setLayout(new FlowLayout());

frameObj.setSize(300, 300);
frameObj.setVisible(true);
}

// main method
public static void main(String args[])
{
    new FlowLayoutExample();
}
}

```

Output:



Example of FlowLayout class: Using FlowLayout(int align) constructor

FileName: MyFlowLayout.java

```

import java.awt.*;
import javax.swing.*;

public class MyFlowLayout{
    JFrame f;
    MyFlowLayout(){
        f=new JFrame();

        JButton b1=new JButton("1");
        JButton b2=new JButton("2");
        JButton b3=new JButton("3");
        JButton b4=new JButton("4");
        JButton b5=new JButton("5");

        // adding buttons to the frame
        f.add(b1); f.add(b2); f.add(b3); f.add(b4); f.add(b5);

        // setting flow layout of right alignment
    }
}

```

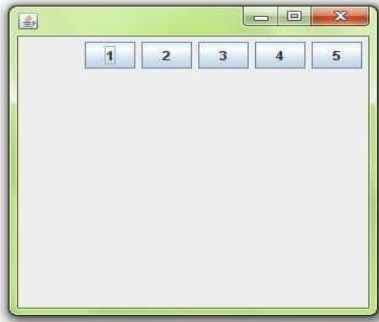
```

        f.setLayout(new FlowLayout(FlowLayout.RIGHT));

        f.setSize(300,300);
        f.setVisible(true);
    }
    public static void main(String[] args) {
        new MyFlowLayout();
    }
}

```

Output:



Example of FlowLayout class: Using FlowLayout(int align, int hgap, int vgap) constructor

FileName: FlowLayoutExample1.java

// import statement

import java.awt.*;

import javax.swing.*;

```

public class FlowLayoutExample1
{
    JFrame frameObj;

```

// constructor

FlowLayoutExample1()

```
{
```

// creating a frame object

frameObj = new JFrame();

// creating the buttons

JButton b1 = new JButton("1");

JButton b2 = new JButton("2");

JButton b3 = new JButton("3");

JButton b4 = new JButton("4");

JButton b5 = new JButton("5");

JButton b6 = new JButton("6");

JButton b7 = new JButton("7");

JButton b8 = new JButton("8");

JButton b9 = new JButton("9");

JButton b10 = new JButton("10");

```

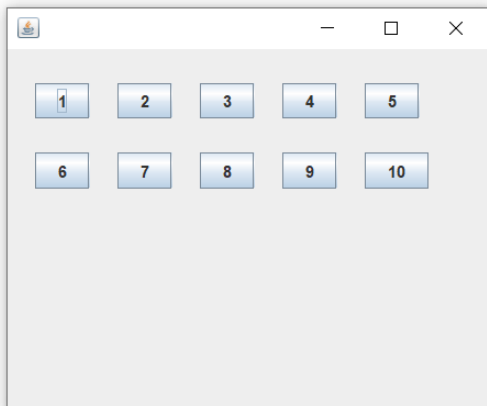
// adding the buttons to frame
frameObj.add(b1); frameObj.add(b2); frameObj.add(b3); frameObj.add(b4);
frameObj.add(b5); frameObj.add(b6); frameObj.add(b7); frameObj.add(b8);
frameObj.add(b9); frameObj.add(b10);

// parameterized constructor is used
// where alignment is left
// horizontal gap is 20 units and vertical gap is 25 units.
frameObj.setLayout(new FlowLayout(FlowLayout.LEFT, 20, 25));

frameObj.setSize(300, 300);
frameObj.setVisible(true);
}
// main method
public static void main(String args[])
{
    new FlowLayoutExample1();
}
}

```

Output:



Additional Example for combined Layouts

```

import java.applet.Applet;
import java.awt.*;
// import java.awt.event.*;

/*<applet code= layout width=500 height=500></applet>*/public
class layout extends Applet
{
    public void init()
    {

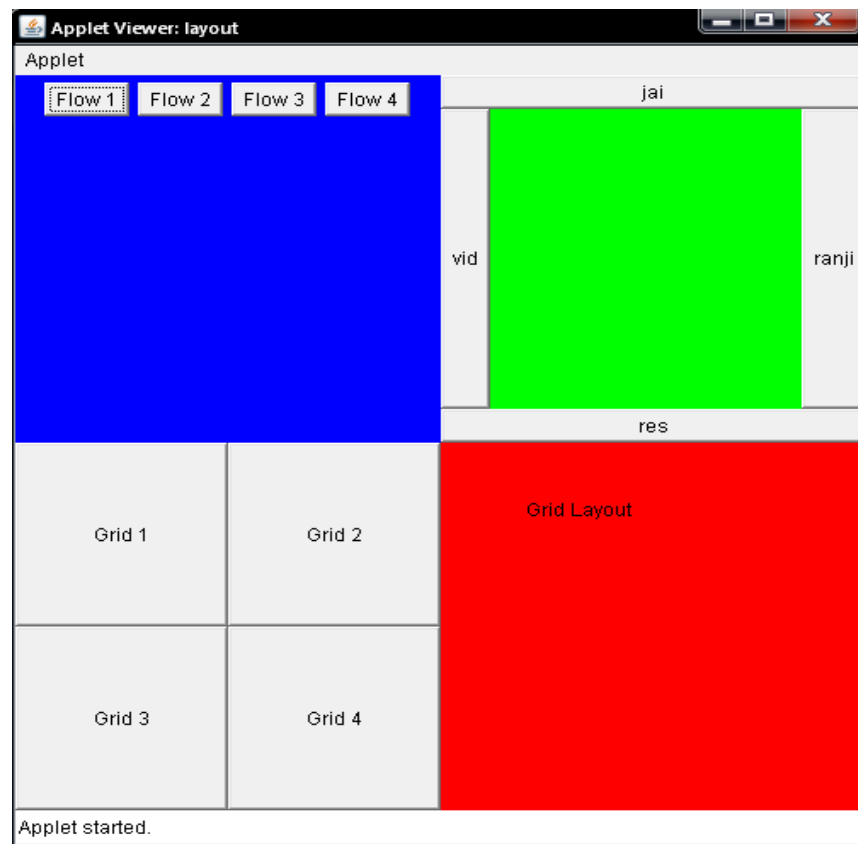
```

```
setLayout(new GridLayout(2,2)); Panel
flow=new Panel(); flow.setLayout(new
FlowLayout());
flow.setBackground(Color.blue);
flow.add(new Button("Flow 1"));
flow.add(new Button("Flow 2"));
flow.add(new Button("Flow 3"));
flow.add(new Button("Flow 4"));
add(flow);
Panel border=new Panel();
border.setLayout(new BorderLayout());
border.setBackground(Color.green);
border.add(new Button("jai"),BorderLayout.NORTH);
border.add(new Button("res"),BorderLayout.SOUTH);
border.add(new Button("ranji"),BorderLayout.EAST);
border.add(new Button("vid"),BorderLayout.WEST);
add(border);
Panel grid=new Panel();
grid.setLayout(new GridLayout(2,2));
grid.setBackground(Color.red);
grid.add(new Button("Grid 1"));
grid.add(new Button("Grid 2"));
grid.add(new Button("Grid 3"));
grid.add(new Button("Grid 4"));
add(grid);
}
public void paint(Graphics g)
{
setBackground(Color.red);
g.drawString("Grid Layout",300,300);
```

```
}  
  
}
```

```
I:\jdk1.6.0\bin>javac layout.java  
  
I:\jdk1.6.0\bin>appletviewer layout.java  
  
I:\jdk1.6.0\bin>
```

OUTPUT



Java AWT MenuItem and Menu

The object of MenuItem class adds a simple labeled menu item on menu. The items used in a menu must belong to the MenuItem or any of its subclass.

The object of Menu class is a pull down menu component which is displayed on the menu bar. It inherits the MenuItem class.

AWT MenuItem class declaration

public class MenuItem **extends** MenuComponent **implements** Accessible

AWT Menu class declaration

public class Menu **extends** MenuItem **implements** MenuContainer, Accessible

Java AWT MenuItem and Menu Example

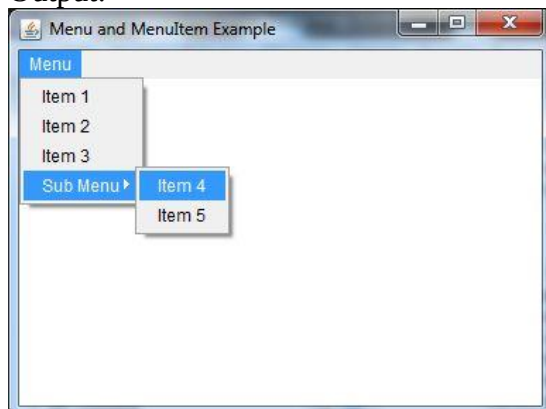
import java.awt.*;

```

class MenuExample
{
    MenuExample(){
        Frame f= new Frame("Menu and MenuItem Example");
        MenuBar mb=new MenuBar();
        Menu menu=new Menu("Menu");
        Menu submenu=new Menu("Sub Menu");
        MenuItem i1=new MenuItem("Item 1");
        MenuItem i2=new MenuItem("Item 2");
        MenuItem i3=new MenuItem("Item 3");
        MenuItem i4=new MenuItem("Item 4");
        MenuItem i5=new MenuItem("Item 5");
        menu.add(i1);
        menu.add(i2);
        menu.add(i3);
        submenu.add(i4);
        submenu.add(i5);
        menu.add(submenu);
        mb.add(menu);
        f.setMenuBar(mb);
        f.setSize(400,400);
        f.setLayout(null);
        f.setVisible(true);
    }
    public static void main(String args[])
    {
        new MenuExample();
    }
}

```

Output:



Java AWT PopupMenu

PopupMenu can be dynamically popped up at specific position within a component. It inherits the Menu class.

AWT PopupMenu class declaration

public class PopupMenu **extends** Menu **implements** MenuContainer, Accessible

Java AWT PopupMenu Example

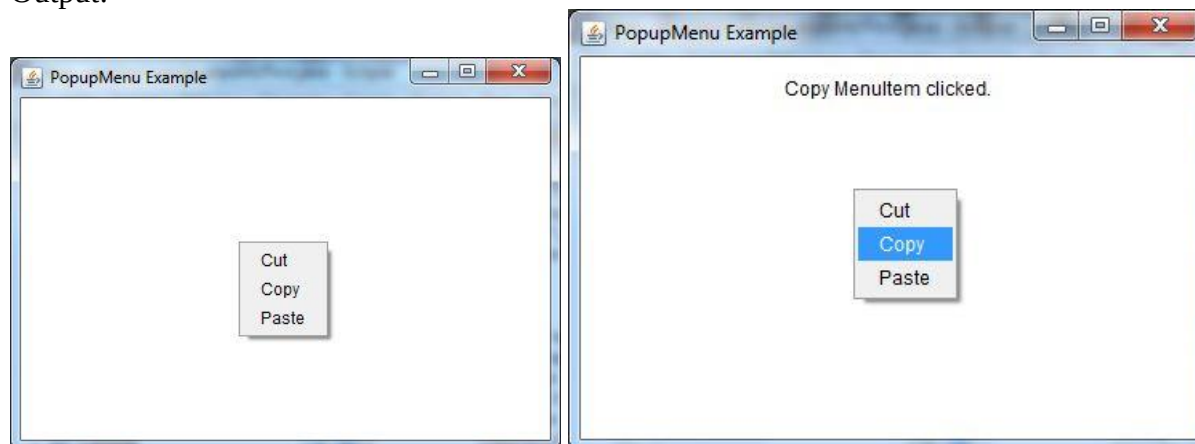
import java.awt.*;

```

import java.awt.event.*;
class PopupMenuExample
{
    PopupMenuExample(){
        final Frame f= new Frame("PopupMenu Example");
        final PopupMenu popupmenu = new PopupMenu("Edit");
        MenuItem cut = new MenuItem("Cut");
        cut.setActionCommand("Cut");
        MenuItem copy = new MenuItem("Copy");
        copy.setActionCommand("Copy");
        MenuItem paste = new MenuItem("Paste");
        paste.setActionCommand("Paste");
        popupmenu.add(cut);
        popupmenu.add(copy);
        popupmenu.add(paste);
        f.addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent e) {
                popupmenu.show(f , e.getX(), e.getY());
            }
        });
        f.add(popupmenu);
        f.setSize(400,400);
        f.setLayout(null);
        f.setVisible(true);
    }
    public static void main(String args[])
    {
        new PopupMenuExample();
    }
}

```

Output:



Java AWT Panel

The Panel is a simplest container class. It provides space in which an application can attach any other component. It inherits the Container class.

It doesn't have title bar.

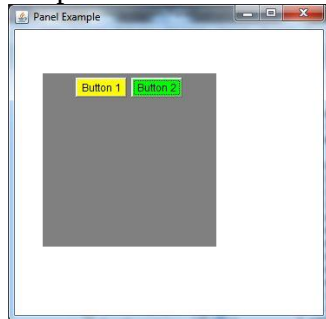
AWT Panel class declaration

public class Panel extends Container implements Accessible

Java AWT Panel Example

```
import java.awt.*;
public class PanelExample {
    PanelExample()
    {
        Frame f= new Frame("Panel Example");
        Panel panel=new Panel();
        panel.setBounds(40,80,200,200);
        panel.setBackground(Color.gray);
        Button b1=new Button("Button 1");
        b1.setBounds(50,100,80,30);
        b1.setBackground(Color.yellow);
        Button b2=new Button("Button 2");
        b2.setBounds(100,100,80,30);
        b2.setBackground(Color.green);
        panel.add(b1); panel.add(b2);
        f.add(panel);
        f.setSize(400,400);
        f.setLayout(null);
        f.setVisible(true);
    }
    public static void main(String args[])
    {
        new PanelExample();
    } }
```

Output:



Java AWT Dialog

The Dialog control represents a top level window with a border and a title used to take some form of input from the user. It inherits the Window class.

Unlike Frame, it doesn't have maximize and minimize buttons.

Frame vs Dialog

Frame and Dialog both inherits Window class. Frame has maximize and minimize buttons but Dialog doesn't have.

AWT Dialog class declaration

```
public class Dialog extends Window
```

Java AWT Dialog Example

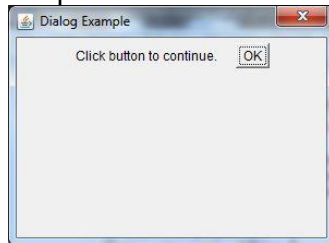
```
import java.awt.*;
import java.awt.event.*;
public class DialogExample {
```

```

private static Dialog d;
DialogExample() {
    Frame f= new Frame();
    d = new Dialog(f , "Dialog Example", true);
    d.setLayout( new FlowLayout() );
    Button b = new Button ("OK");
    b.addActionListener ( new ActionListener()
    {
        public void actionPerformed((ActionEvent e )
        {
            DialogExample.d.setVisible(false);
        }
    });
    d.add( new Label ("Click button to continue."));
    d.add(b);
    d.setSize(300,300);
    d.setVisible(true);
}
public static void main(String args[])
{
    new DialogExample();
} }

```

Output:



Additional Examples

WORKING WITH DIALOGS AND MENUS

```
import java.awt.*; //for Graphics, Frame classes and various controls
```

```
import java.awt.event.ActionListener;
```

```
import java.awt.event.ActionEvent;
```

```
class myframe extends Frame implements ActionListener
```

```
{
```

```
    MenuBar mb;
```

```
    Menu graphics;
```

```
    MenuItem cir,rect,line,exit;
```

```
    Dialog d;
```

```
    Label l; Button
```

```
    b1,b2;int flag;
```

```

myframe()
{
    mb=new MenuBar(); graphics=new
    Menu("Graphical"); cir=new
    MenuItem("Oval"); rect=new
    MenuItem("Rect"); line=new
    MenuItem("Line"); exit=new
    MenuItem("Exit"); graphics.add(cir);
    graphics.add(rect); graphics.add(line);
    graphics.add(exit); mb.add(graphics);
    rect.addActionListener(this);
    cir.addActionListener(this);
    line.addActionListener(this);
    exit.addActionListener(this);
    setMenuBar(mb);
    d=new Dialog(this, "Request Window", true); l=new
    Label("Do you want to exit");
    d.setBackground(Color.cyan);
    b1=new Button("yes");
    b2=new Button("no");
    l.setBounds(30,30,120,30);
    b1.setBounds(30,70,40,20);
    b2.setBounds(30,70,40,20);
    d.add(l);
    d.add(b1);
    d.add(b2);
    b1.addActionListener(this);
    b2.addActionListener(this);
}
public void actionPerformed(ActionEvent ae)
{
    if(ae.getSource()==cir)
        flag=1;
    if(ae.getSource()==rect)
        flag=2;
    if(ae.getSource()==line)
        flag=3; if(flag>0&&flag<4)
        repaint();
    if(ae.getSource()==exit)
    {
        d.setBounds(100,100,200,130);
        d.setVisible(true);
    }
}

```

```

    }
    if(ae.getSource()==b1)System.exit(0);
    if(ae.getSource()==b2)
        d.setVisible(false);
    }
    public void paint(Graphics g)
    {
        if(flag==1)
            g.drawOval(90,100,60,60);
        if(flag==2)
            g.drawRect(90,100,150,60);
        if(flag==3)
            g.drawLine(30,100,150,60);
    }
}
public class menuDemo
{
    public static void main(String args[])
    {
        myframe m=new myframe();
        m.setVisible(true);
        m.setBackground(Color.pink);
        m.setSize(400,400);
    }
}

```

OUTPUT

